Compiler Warnings and clang-format

Catch bugs at compile time and keep your C++ style consistent

Part 1: Compiler Warnings

1.1 What they are

The compiler can tell you when code is **legal but suspicious**. It still compiles, but it prints a warning pointing at a likely bug. Warnings are free, instant, and need no running program, so they catch bugs before you ever test anything.

By default, GCC and Clang show very few warnings. You have to turn the rest on.

1.2 The flags

```
g++ -Wall -Wextra -Wpedantic -g main.cpp -o main
```

- `-Wall`: a core set of common problems (unused variables, missing return, suspicious comparisons)
- `-Wextra`: more checks on top of `-Wall`
- `-Wpedantic`: warns when you use non-standard C++ extensions

Good extras:

- `-Wshadow`: a local variable hides another with the same name
- `-Wconversion`: risky implicit conversions (e.g. `double` to `int`, `long` to `int`)
- `-Wformat=2`: stricter `printf`-style format checks
- `-Wnull-dereference`: possible null pointer use

Make warnings fatal:

- `-Werror`: any warning becomes a compile error. Use it in CI so warnings cannot pile up, but not while learning.

1.3 Example

```
#include <iostream>

int get(int x) {
    if (x > 0) return 1;
}                               // bug: no return on some paths

int main() {
    int unused = 5;             // bug: never used
    int n = 3;
    unsigned int u = 10;
    if (n < u) std::cout << "hi\n";   // bug: signed/unsigned comparison
    return 0;
}
```

Compile with `g++ -Wall -Wextra main.cpp` and you will get roughly:

```
main.cpp:5:1: warning: control reaches end of non-void function [-Wreturn-type]
main.cpp:8:9: warning: unused variable 'unused' [-Wunused-variable]
main.cpp:11:10: warning: comparison of integer expressions of different signedness
    [-Wsign-compare]
```

1.4 How to read one

`file:line:column: warning: <message> [-Wflag-name]`

- The **line** is where the compiler is suspicious
- The **`[-Wflag-name]`** at the end tells you which flag triggered it, handy for looking it up or silencing it
- The missing-return warning is a real one: that function has undefined behavior, which UBSan would also flag at runtime

1.5 Tips

- **Treat warnings as bugs.** Fix them instead of ignoring them; a clean build makes the one new warning obvious.
- **Optimization changes warnings.** Some (like possibly-uninitialized variables) only appear at `-O2`, so compile with your release level occasionally.
- **Static analysis goes further.** `g++ -fanalyzer` (GCC) and `clang-tidy` find deeper problems such as leaks and null dereferences across functions. Worth knowing they exist; skip them for now.
- **Default for every project:** `-Wall -Wextra -Wpedantic` from day one.

1.6 Cheat sheet

Goal	Flag
Standard good set	`-Wall -Wextra -Wpedantic`
Catch shadowed variables	`-Wshadow`
Catch risky conversions	`-Wconversion`
Fail the build on warnings	`-Werror`
Deeper analysis	`-fanalyzer` or `clang-tidy`

Together with the sanitizers this gives you the whole "catch bugs early" workflow: **warnings at compile time, ASan/UBSan at runtime**.

Part 2: clang-format

2.1 What it is

clang-format is a tool that **automatically rewrites your C/C++ code into a consistent style**: indentation, spacing, brace placement, line wrapping, and so on. It does not change what your code does, only how it looks.

It exists so nobody argues about style. In a team, everyone runs the same formatter, so code reviews are about logic instead of where the brace goes.

2.2 Install

```
sudo apt install clang-format      # Linux / WSL
clang-format --version
```

2.3 Basic use

```
clang-format main.cpp              # print the formatted version (file unchanged)
clang-format -i main.cpp           # format the file in place
clang-format -i *.cpp *.h          # format many files
```

- `-i`: edit the file **in place** (without it, the result is only printed)
- `-style=Google` (or `LLVM`, `Mozilla`, `WebKit`, `Chromium`): pick a built-in style for one run

2.4 Example

Before:

```
int sum(int*arr,int n){
int total=0;
for(int i=0;i<n;i++){total+=arr[i];}
return total;}
```

After `clang-format -i`:

```
int sum(int *arr, int n) {
  int total = 0;
  for (int i = 0; i < n; i++) {
    total += arr[i];
  }
  return total;
}
```

2.5 Choosing your style: the .clang-format file

Put a file named `.clang-format` in your project root, and clang-format uses it automatically for every file below that folder. Generate a starting point:

```
clang-format -style=llvm -dump-config > .clang-format
```

Or write a short one by hand:

```
BasedOnStyle: Google
IndentWidth: 4
ColumnLimit: 100
```

- `BasedOnStyle`: start from a known style and only override what you want
- `IndentWidth`: spaces per indent level

- `ColumnLimit`: the maximum line length before it wraps

2.6 Use it automatically in VS Code

1. Install the **C/C++** extension (clang-format is built in)
2. Open Settings and turn on **Format on Save** (`"editor.formatOnSave": true`)
3. Put a `.clang-format` file in your project

Now every save formats the file. Manual format is `Shift+Alt+F` (Windows/Linux).

2.7 Other useful bits

Turn it off for a block (for example a hand-aligned table):

```
// clang-format off
int matrix[2][2] = {{1,  0},
                    {0,  1}};
// clang-format on
```

Check formatting without changing files (good for CI). This fails if the file is not formatted, so unformatted code cannot be merged:

```
clang-format --dry-run -Werror main.cpp
```

Pre-commit hook: a hook that runs clang-format before each commit keeps the whole repo consistent.

2.8 Why it matters for you

Big C++ projects, including llama.cpp, ship their own `.clang-format`. If you contribute a pull request, run the formatter first or reviewers will ask you to. Formatting your own project this way also makes your portfolio code look professional.

2.9 Cheat sheet

Goal	Command
Format a file in place	`clang-format -i main.cpp`
Preview without changing	`clang-format main.cpp`
Pick a built-in style	`-style=Google`
Create a config file	`clang-format -style=llvm -dump-config > .clang-format`
CI check	`clang-format --dry-run -Werror main.cpp`
Skip a section	`// clang-format off` ... `// clang-format on`